

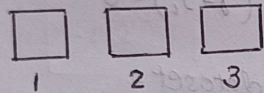
Gradient Boosting

Intuition:

⇒ In case of regression problem when we go for the gradient boosting the very first trained model will give output as the mean of original output column.

iq	cgpa	salary
90	8	3
100	7	4
110	6	8
120	9	6
80	5	3

∴ models be:



Also, mean of salary = prediction of model 1

$$= \frac{3+4+8+6+3}{5} = 4.8$$

Thus, dataset updated now to:

iq	cgpa	salary	Pred1
90	8	3	4.8
100	7	4	4.8
110	6	8	4.8
120	9	6	4.8
80	5	3	4.8

Now what about Performance of model 1 (model 1) we calc. it using pseudo-residual.

∴ pseudo residual = actual - predicted

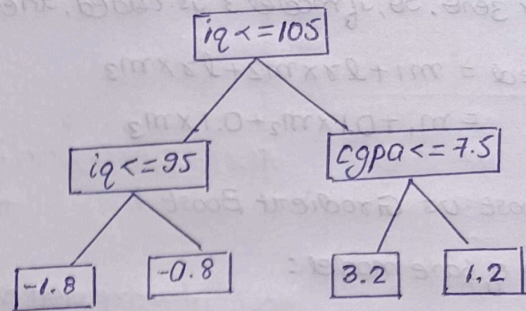
Thus, dataset updated now to:

iq	cgpa	salary	Pred1	res1
90	8	3	4.8	-1.8
100	7	4	4.8	-0.8
110	6	8	4.8	3.2
120	9	6	4.8	1.2
80	5	3	4.8	-1.8

Now, how to send these mistakes to model 2? we will use Decision Tree as our model 2. And for

that we will give 'iq', 'cgpa' as input and output be 'res1'.

∵ model 2 is not predicting salary, it is calculating mistakes made by model 1. we get decision tree as:



Thus, dataset updated now to:

iq	cgpa	salary	Pred1	res1	pred2
90	8	3	4.8	-1.8	-1.8
100	7	4	4.8	-0.8	-0.8
110	6	8	4.8	3.2	3.2
120	9	6	4.8	1.2	1.2
80	5	3	4.8	-1.8	-1.8

Now, suppose we need to predict using model 1 and model 2 only. then in that case to avoid overfitting we multiply prediction of model 2 with a learning rate.

$$\% \hat{y}_{pred} = m_1 + lr \times m_2$$

where, $lr = 0.1$ (generally)

∴ for row 2:

$$pred = m_1 + lr \times m_2 = 3 + 0.1 \times (-1.8) = 4.7$$

But, actual output of row 2 is 3 and predicted is 4.7, too different. So, we will use one another model after model 2 to minimise this difference. Now for model 3, we will calculate pseudo residual as, $res_2 = \text{actual} - \hat{y}_{pred}$ (for each row)

Thus, dataset updated to:

iq	cgpa	ipa	Pred1	res1	pred2	res2
90	8	3	4.8	-1.8	-1.8	-1.62
100	7	4	4.8	-0.8	-0.8	-0.72
110	6	8	4.8	3.2	3.2	2.88
120	9	6	4.8	1.2	1.2	1.08
80	5	3	4.8	-1.8	-1.8	-1.62

Clearly, in ideal case we need residual to be zero. And here, $r_{m2} < r_{m1}$. So if we add more subsequent models we will be able to get near zero. So, if model 3 is added, then

$$y_{\text{pred}} = m_1 + 0.1 \times m_2 + 0.1 \times m_3$$

$$= m_1 + 0.1 \times m_2 + 0.1 \times m_3$$

$\Rightarrow$ AdaBoost vs Gradient Boost:

① Type of base model:

AdaBoost $\rightarrow$ Decision Stump
(max depth = 1)

Gr.B. $\rightarrow$ Large/shallow decision tree
(8-32)

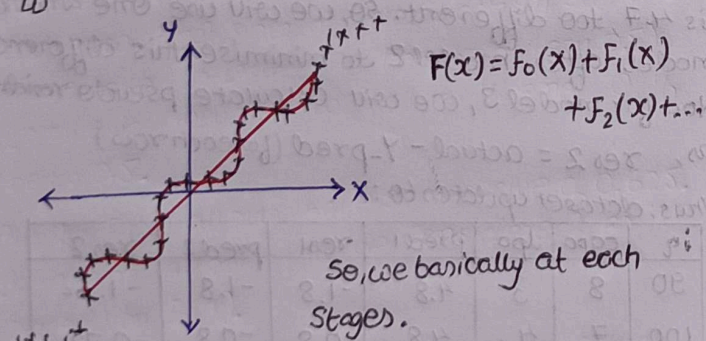
② Learning Rate:

AdaBoost $\rightarrow$ diff. model get diff. wt.

Gr.B. $\rightarrow$ same learning rate is used here.

Maths of Gradient Boosting (for regression)

$\Rightarrow$ What is Additive Modelling? Here, we consider that a final function is actually sum of some smaller functions. For a dataset which is non-linear as well as not applicable for any other models like poly. regression to cover up, we basically then use combination of other/diff. models, this is called additive modelling.



$\Rightarrow$ Maths:

An input we have a training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$ and M be the no. of iterations.

$$F(x) = f_0(x) + f_1(x) + f_2(x) + f_3(x) + \dots + f_N(x)$$

$$\text{Step 1: } f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$$

$$\therefore L = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\therefore f_0(x) = \arg \min_{\gamma} \frac{1}{2} \sum_{i=1}^n (y_i - \gamma)^2$$

So, for min. value,

$$\frac{d f_0(x)}{d \gamma} = \frac{d}{d \gamma} \frac{1}{2} \sum_{i=1}^n (y_i - \gamma)^2 = 0$$

$$\Rightarrow \sum_{i=1}^n (\gamma - y_i) = 0$$

Say, we have dataset:

R&D spend	Admin	Marketing spend	Profit
165.0	137.0	472.0	192.0
101.0	92.0	250.0	144.0
29.0	127.0	201.0	91.0

$$\therefore \sum_{i=1}^3 (\gamma - y_i) = 0 \Rightarrow (\gamma - 192) + (\gamma - 144) + (\gamma - 91) = 0$$

$$\therefore \gamma = \frac{192 + 144 + 91}{3} = \text{mean of } Y \text{ (output) column}$$

Step 2: For $m=1$ to M :

① For $i=1, 2, \dots, N$ compute:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

where, i = rows no.

m = no. of decision trees

So, basically we are calculating r (residual) for each row of each decision tree.

For $m=1$,

$$\gamma_{i1} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_0}$$

∵ we had 3 rows in dataset, we will calculate γ_{i1} as: $\gamma_{11}, \gamma_{21}, \gamma_{31}$.

$$\hat{y}_i = f(x_i)$$

$$\therefore \gamma_{i1} = - \left[\frac{\partial}{\partial \hat{y}_i} L(y_i, \hat{y}_i) \right]_{f=f_0}$$

$$= - \left[\frac{\partial}{\partial \hat{y}_i} \frac{1}{2} (y_i - \hat{y}_i)^2 \right]_{f=f_0} = \left[(y_i - \hat{y}_i) \right]_{f=f_0}$$

$$= \left[(y_i - f(x_i)) \right]_{f=f_0} = y_i - f_0(x_i)$$

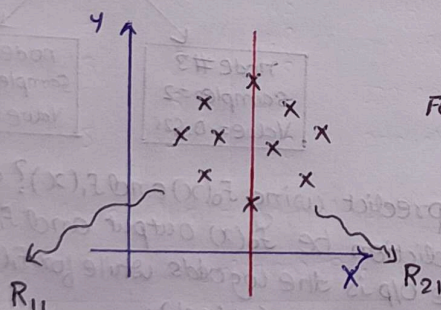
$$\therefore \gamma_{11} = y_1 - f_0(x_1) = 192 - 142$$

$$\gamma_{21} = y_2 - f_0(x_2) = 144 - 142$$

$$\gamma_{31} = y_3 - f_0(x_3) = 91 - 142$$

⑥ Fit a regression tree to the target y giving terminal regions R_{jm} , $j=1, 2, \dots, J_m$.
But what's this terminal region? It's the sections in which tree or data is divided for each tree.

Suppose our dataset be like this:

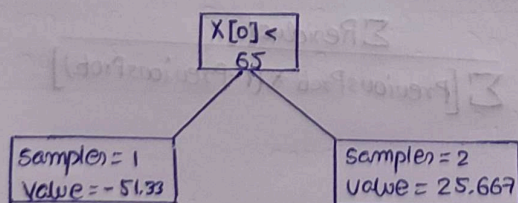


For $m=1$

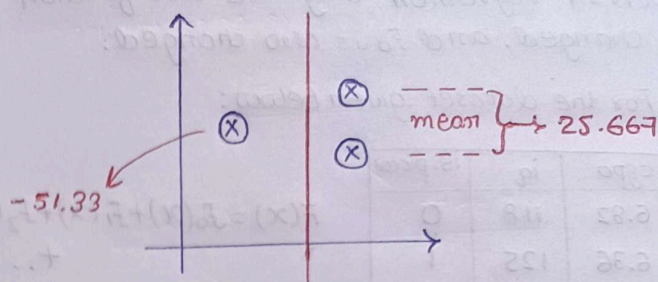
(first terminal region of first decision tree)

(2nd terminal region of first decision tree)

∵ our decision tree is:



∵ for our dataset, the split is:



⑦ For $j=1, 2, 3, \dots, J_m$ compute:

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

$$\therefore \gamma_{j1} \Rightarrow \gamma_{11} = \arg \min_{\gamma} \sum_{x_i \in R_{11}} L(y_i, f_{m-1}(x_i) + \gamma)$$

$$\begin{matrix} \downarrow & \downarrow \\ \gamma_{11} & \gamma_{21} \end{matrix}$$

$$\therefore \gamma_{11} = \arg \min_{\gamma} \frac{1}{2} (y_i - (f_0(x_i) + \gamma))^2$$

$$\therefore \frac{dL}{d\gamma} = 0 \Rightarrow y_i - f_0(x_i) - \gamma = 0$$

$$\therefore \gamma_{11} = 91 - 142 - \gamma = 0 \Rightarrow \gamma = -51$$

Similarly for another region,

$$\gamma = 25$$

$$\textcircled{d} \text{ update } f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$$

$$\therefore f_1(x) = f_0(x) + \text{decision tree output}$$

$$f_2(x) = f_1(x) + \text{" " 2 output}$$

$$f_3(x) = f_2(x) + \text{" " 3 "}$$

$$\vdots \quad \quad \quad \vdots$$

Step 3: output $\hat{f}(x) = f_m(x)$.

Gradient Boosting ∵ (for classification)

⇒ All steps basically are same as which we used in regression, only the loss function is changed, and f_0 is also changed.

⇒ For the dataset given below:

cgpa	iq	is-placed
6.82	118	0
6.36	125	1
5.39	99	1
5.50	106	1
6.35	148	0
9.13	148	1
7.17	147	1
7.72	72	0

$$F(x) = f_0(x) + f_1(x) + f_2(x) + \dots$$

Now, in regression case for $f_0(x)$ we had used mean of output column. But, for classification, this will not be a good idea.

So, we will go with $\log(\text{odds})$, where,

$$\log(\text{odds}) = \log\left(\frac{\text{No. of } 1}{\text{No. of } 0}\right)$$

$$\therefore \log(\text{odds}) = \log\left(\frac{5}{3}\right) = f_0(x) = 0.51$$

for each rows.

Now, we need to calculate the diff. b/w actual output and $f_0(x)$ value. But "is-placed" column is of probability and $f_0(x)$ is $\log(\text{odds})$, so we need to convert $\log(\text{odds})$ into probability as well.

$$\text{Probability } P = \frac{1}{1 + e^{-\log(\text{odds})}} = \frac{1}{1 + e^{-0.51}} = 0.625$$

∴ the $f_0(x)$ model gives output as 0.625 for each row. And, since it's classification scenario, so we take a value as threshold (say 0.5), if value is above it we call it '1' and '0' if below 0.5.

But, for $f_0(x)$ all rows have same value of 0.625. It means all rows will be considered as '1' (placed) by $f_0(x)$.

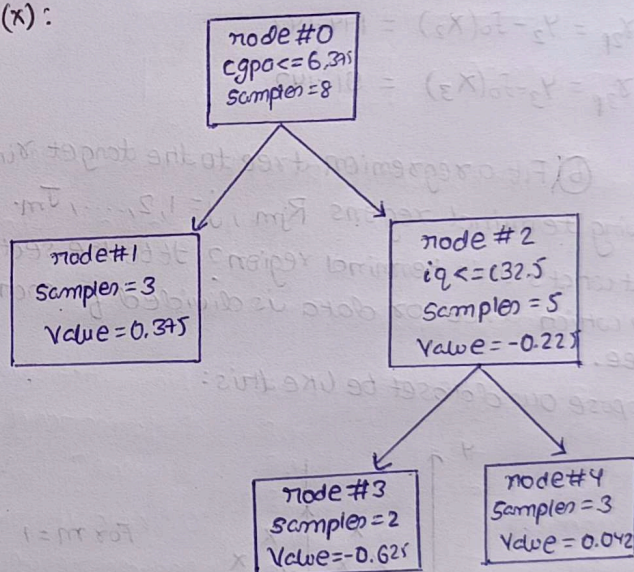
Now, we will calculate the residual as actual - predicted, say res_1 .

So, data can now be seen as:

cgpa	iq	is-placed	pre1 (logodds)	pre1 (Probability)	res1
6.82	118	0	0.51	0.625	-0.625
6.36	125	1	0.51	0.625	0.375
5.39	99	1	0.51	0.625	0.375
5.50	106	1	0.51	0.625	0.375
6.35	148	0	0.51	0.625	-0.625
9.13	148	1	0.51	0.625	0.375
7.17	147	1	0.51	0.625	0.375
7.72	72	0	0.51	0.625	-0.625

Now, a decision tree (a regression tree) is trained as a weak learner for cgpa , iq as input and res_1 as the output.

$f_1(x)$:



But, how to predict using $f_0(x)$ and $f_1(x)$? The overall prediction be $f_0(x)$ output and $f_1(x)$ output. For $f_0(x)$ the O/P is the $\log(\text{odds})$ while for $f_1(x)$ the output be the probability (at leaf). ∴ we need to convert the prob. to $\log(\text{odds})$ for each node, using:

$$\frac{\sum \text{Residual}}{\sum [\text{Previous Prob} * (1 - \text{Previous Prob})]}$$

Now, we will decide which point goes in which node.

Say for Node 3:

$$\frac{-0.625 - 0.625}{0.625(1-0.625) + 0.625(1-0.625)} = \frac{-1}{0.375} = -2.66$$

lly, we will calculate for each point of each leaf node.

∴ log odds for node 1 = 1.6

" " node 2 = -2.66

" " node 3 = 0.18

∴ we know, prediction be $f_0(x) + f_1(x)$ for each row.

∴ as of now, our dataset be:

cgo	iq	is-placed	Pre1 (log odds)	Pre1 (prob.)	res1	leaf-entry1
			0.51			3
			0.51			1
			0.51			1
			0.51			4
			0.51			4
			0.51			4
			0.51			4
			0.51			3

This pre2 is log odd sum of $f_0(x)$ and $f_1(x)$. so, we again need to convert it into ^{Prob} log odds form, using

$$P = \frac{1}{1 + e^{-\text{log odds}}}$$

This resultant prob. of the log odds of Pre2 will give the predictive result of both $f_0(x)$ and $f_1(x)$.

Now, we will calculate the residual of combined $f_0(x)$ and $f_1(x)$, and then train another decision tree $f_2(x)$.

Pre2 (log odds)
-2.159
2.110
2.110
2.110
0.650
0.690
0.690
-2.159